HW7 Part C: Two Textures Lab Report

CS 5382 Computer Graphics | Daniel Oni | Spring 2026

What the Program Does

The program runs two textures at the same time and combines them per fragment. Texture unit 0 holds the live webcam feed, which is refreshed every render call. Texture unit 1 holds a procedurally generated 64 by 64 black and white checkerboard that is built once at startup from a float array and converted to an unsigned byte array before upload. The fragment shader samples both textures at the same uv coordinate and builds the output color by pulling the red channel from the webcam and the green and blue channels from the checkerboard.

What I Changed to Make This Work

1. Added a second sampler and a second texture object. The fragment shader was extended with a second sampler2D uniform, and the JavaScript side got a matching texture2 variable along with a textureloc2 uniform location. Each sampler is pointed at its own texture unit with gl.uniform1i.

```
uniform sampler2D texturevid;     // webcam, unit 0
uniform sampler2D texturevid2;    // checkerboard, unit 1

textureloc  = gl.getUniformLocation(program, "texturevid");
gl.uniform1i(textureloc,  0);

textureloc2 = gl.getUniformLocation(program, "texturevid2");
gl.uniform1i(textureloc2, 1);
```

2. Dropped in the checkerboard generation code from class. I used the 64 by 64 pattern from the class Two Textures example, which fills a float array based on an XOR of two bit masks and then converts it into a Uint8Array for the texImage2D call.

```
var c = (((i & 0x8) == 0) ^ ((j & 0x8) == 0));
image1[i][j] = [c, c, c, 1];
```

3. Separate texImage2D calls for each texture unit. Texture 1 is allocated with a null source since it gets overwritten by each video frame. Texture 2 is uploaded once from the image2 byte array. I switched gl.activeTexture between the two setups so each bind lands on the right unit.

```
gl.activeTexture(gl.TEXTURE0);
gl.bindTexture(gl.TEXTURE_2D, texture1);
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, 400, 400, 0,
              gl.RGBA, gl.UNSIGNED_BYTE, null);

gl.activeTexture(gl.TEXTURE1);
gl.bindTexture(gl.TEXTURE_2D, texture2);
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, texSize, texSize, 0,
              gl.RGBA, gl.UNSIGNED_BYTE, image2);
```

4. Fragment shader combines channels from the two textures. The output color takes its red channel from tex1 and its green and blue channels from tex2. Because the checkerboard is pure black and pure white, tex2.bg is either (1, 1) or (0, 0), which acts like a mask that turns the video into a red-only image in the dark squares and lets the original webcam brightness show through in the light squares.

```
vec4 tex1 = vec4( texture(texturevid,  vTexCoord).rgb, 1.0 );
vec4 tex2 = vec4( texture(texturevid2, vTexCoord).rgb, 1.0 );
fColor = vec4( tex1.r, tex2.bg, 1.0 );
```

Challenges and How I Resolved Them

Black output on the first couple of frames. The starter passed the video element directly into the first texImage2D call at init time. Because the getUserMedia promise had not resolved yet, the video element had no frames, which caused an INVALID_VALUE warning and a black canvas until the stream came up. The fix was to allocate texture unit 0 with null for the source and then let the render loop fill it in once the video readyState confirms it has a current frame. I added a check for video.readyState before calling texImage2D in render to make sure I never upload from an empty video.

Checkerboard came out mirrored. The video upload uses UNPACK_FLIP_Y_WEBGL set to true so the webcam feed is right side up. The checkerboard does not need that flip because it is already laid out in the row order I want. When the setting was left on globally, the checker pattern appeared reversed top to bottom. The fix was to toggle the pixel store flag off just before the checkerboard upload and turn it back on after, so the setting is only applied to the video frames.

```
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true);   // for video
// ... allocate texture 1 ...
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, false);  // for checkerboard
// ... upload texture 2 ...
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true);   // restore for render loop
```

Making sure each sampler was pointed at the right unit. It is easy to bind the textures correctly and then forget to tell the shader which sampler uses which unit. If both samplers default to 0, both end up reading the same texture and the output stops reacting to one of them. I set uniform1i for both textureloc and textureloc2 explicitly, and I kept the bindings in the same order as the uniform assignments so the mapping stays obvious when I reread the code.

What I Took Away From This

The part that made the most sense to me after this was how much of a compositing tool a fragment shader actually is. Once both textures are bound to units and the samplers know where to look, you can pull individual channels from either source and assemble whatever output color you want. The same structure I built here for video plus checkerboard would work for blending two images, for keying one source against a mask, or for the kind of text overlay patterns the lectures showed with the mask comparison. The interesting work is almost entirely in the shader expression at the end, not in the setup.